



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Crossplane vs Kubebuilder

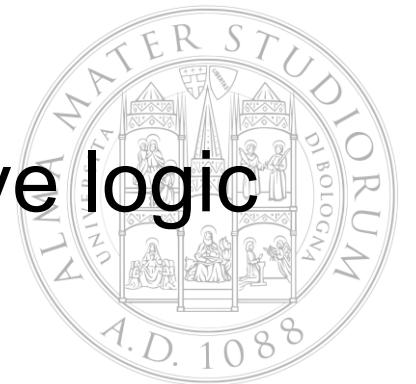
paradigms, abstraction, complexity, extensibility and performance comparison with a common
use case implementation

Infrastructures for Cloud Computing and Big Data M
Lorenzo De Luca
0001240461

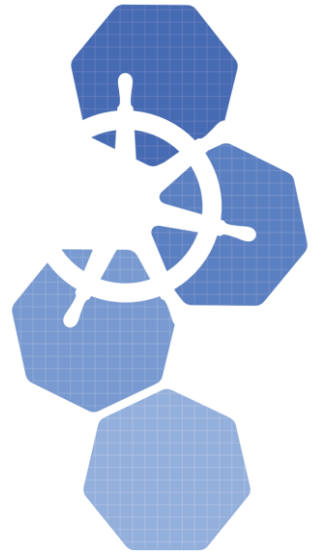
Crossplane



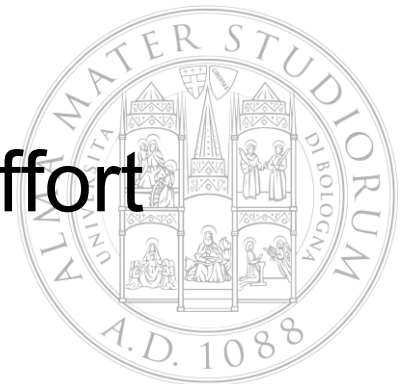
- Universal Control Plane
- Declarative composition model
- Strong for composition and platform engineering
- Logic in YAML + patch/transform functions
- Strength: consistency and reuse
- Limitation: less flexible for complex imperative logic



Kubebuilder



- Low-level Go SDK
- Imperative control model
- Strong for writing specialized controllers from scratch.
- Logic in Go
- Strength: maximum flexibility
- Limitation: higher coding and maintenance effort



Resource Management

Crossplane



Layered model

The user creates a high-level custom resource describing the desired service. Crossplane uses a Composition to generate composed resources, while providers reconcile those resources against Kubernetes or external systems.

Providers

Providers manage external or K8s resources through standard interfaces. They handle authentication, external api calls and controller logic.

Status managed through Crossplane

Ownerships and status aggregation are managed through Crossplane mechanisms

Kubebuilder



Direct control

The controller directly reconciles the Custom Resource and target resources

Custom logic

The reconciler can use any Kubernetes client or external API client

Status managed in code

Ownership references, status conditions, and lifecycle logic are managed in code



Abstraction

Crossplane



High-level Abstraction

User declares the desired state using a custom API

Declare the desired infrastructure

The user only creates a high-level custom resource, the internal resources are generated by the Composition.

Kubebuilder



Low-level framework

Developer defines how the desired state is reached.

Explicit control over lifecycle

The controller directly manages the resources, status updates and error handling.



Extensibility

Crossplane



Extensible through components

Extensible through Providers, Functions, patching, and transformations

Optimized for reusability

Encourages API standardization across teams. Ideal for creating Internal Development Platforms

Easier for standard composition scenarios

The standardized composition model makes it easy to define new platform APIs without writing a dedicated controller.

Kubebuilder



Extensible through coding and external libs

Extensible through arbitrary Go code and external libraries

Optimized for specific domains

Best for complex, conditional, or stateful workflows

Easier for custom logic

Better debuggability in Go due to the concentrated logic.



Which one is better? It depends...

Scenario	Better choice	Reason
Standard infrastructure composition	Crossplane	Declarative and reusable
Internal Developer Platform	Crossplane	User-friendly claims and abstractions
Multi-resource self-service APIs	Crossplane	Composition model fits well
Complex conditional logic	Kubebuilder	Full control in Go
Custom external API integration	Kubebuilder	Any Go library can be used
Fine-grained lifecycle control	Kubebuilder	Reconciliation is fully programmable
Provisioning latency	Kubebuilder	Fewer controller layers
Debuggability	Kubebuilder	Less layers, more control



Kafka Topic Self-Service

<https://github.com/lorenzodeluca/kafka-operator-clash>

Common goal



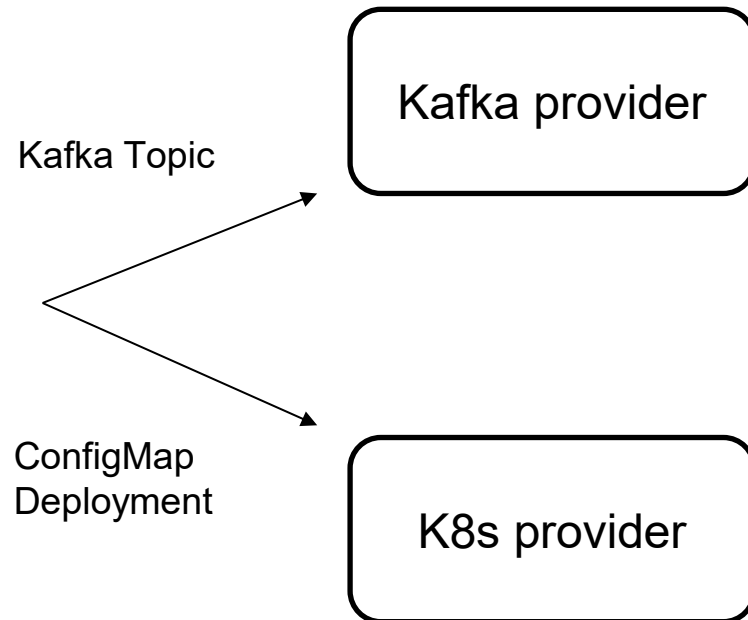
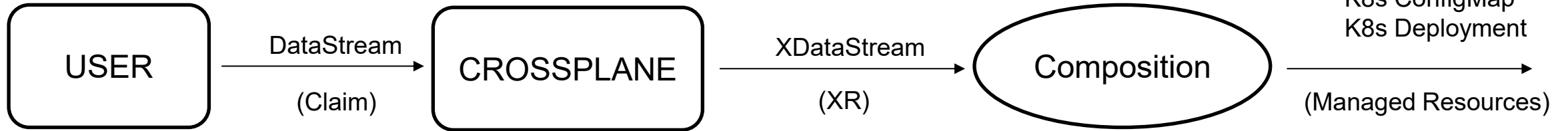
The goal for the implemented operators is to create a new Kafka topic. Instead of manually creating all required resources, the user declares a single object called *DataStream*.

When a *DataStream* is created, the operator must automatically provision and reconcile a ready-to-use data pipeline: a Kafka topic where data can flow, a configuration file with the connection details, and a small consumer application that reads the connection details generated by the operator.

DataStream example:

```
1  apiVersion: messaging.lorenzodeluca.it/v1alpha1
2  kind: DataStream
3  metadata:
4    name: lorenzo-sandbox-stream
5    namespace: default
6  spec:
7    topicName: "user-telemetry-data"
8    partitions: 2
9    replicationFactor: 1
```

Crossplane implementation



api/datastream_xrd.yaml:

```
1  apiVersion: apiextensions.crossplane.io/v1
2  kind: CompositeResourceDefinition
3  metadata:
4    name: xdatastreams.messaging.lorenzodeluca.it
5  spec:
6    group: messaging.lorenzodeluca.it
7    names:
8      kind: XDataStream
9      plural: xdatastreams
10   claimNames:
11     kind: DataStream
12     plural: datastreams
13   versions:
14     - name: v1alpha1
15       served: true
16       referenceable: true
17       schema:
18         openAPIV3Schema:
19           type: object
20           properties:
21             spec:
22               type: object
23               properties:
24                 topicName:
25                   type: string
26                 partitions:
27                   type: integer
28                   default: 1
29                 replicationFactor:
30                   type: integer
31                   default: 1
32               required:
33                 - topicName
```

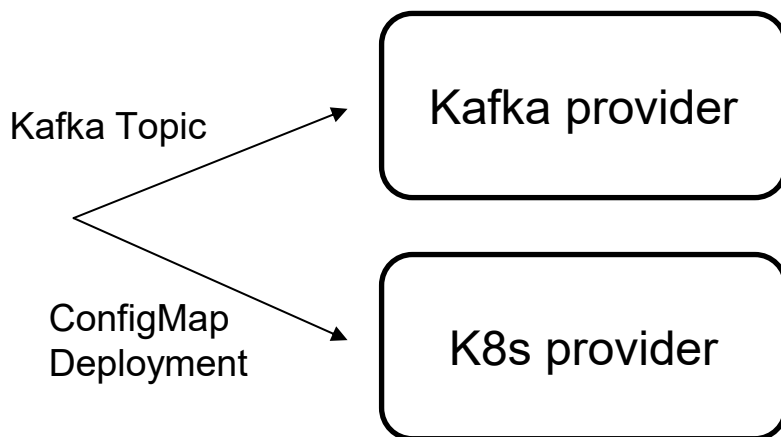


Crossplane – composition

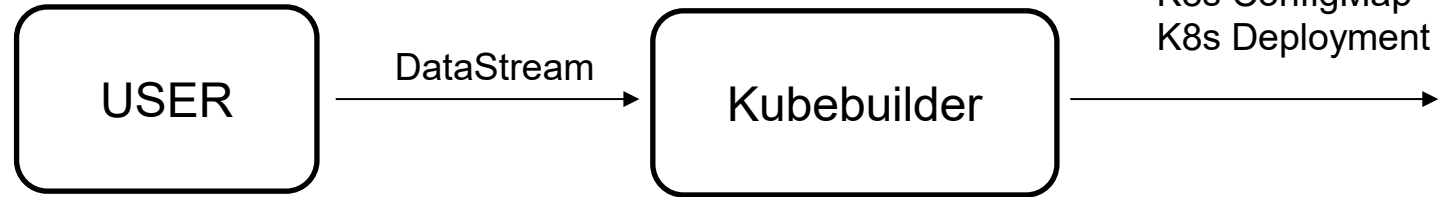
```
- step: patch-and-transform
functionRef:
  name: function-patch-and-transform
input:
  apiVersion: pt.fn.crossplane.io/v1beta1
  kind: Resources
  resources:
    - name: kafka-topic
      base:
        apiVersion: topic.kafka.crossplane.io/v1alpha1
        kind: Topic
        spec:
          providerConfigRef:
            name: kafka-default
          forProvider:
            partitions: 1
            replicationFactor: 1
      patches:
        - type: FromCompositeFieldPath
          fromFieldPath: spec.topicName
          toFieldPath: metadata.name
        - type: FromCompositeFieldPath
          fromFieldPath: spec.partitions
          toFieldPath: spec.forProvider.partitions
        - type: FromCompositeFieldPath
          fromFieldPath: spec.replicationFactor
          toFieldPath: spec.forProvider.replicationFactor
```

```
- name: connection-configmap
base:
  apiVersion: kubernetes.crossplane.io/v1alpha2
  kind: Object
  spec:
    providerConfigRef:
      name: kubernetes-default
    forProvider:
      manifest:
        apiVersion: v1
        kind: ConfigMap
        metadata:
          namespace: default
          name: placeholder
        data:
          KAFKA_BROKER: "kafka.kafka.svc.cluster.local:9092"
          KAFKA_TOPIC: ""
    patches:
      - type: FromCompositeFieldPath
        fromFieldPath: spec.topicName
        toFieldPath: spec.forProvider.manifest.metadata.name
      transforms:
        - type: string
          string:
            type: Format
            fmt: "%s-connection"
      - type: FromCompositeFieldPath
        fromFieldPath: spec.topicName
        toFieldPath: spec.forProvider.manifest.data.KAFKA_TOPIC
```

```
- name: consumer-deployment
base:
  apiVersion: kubernetes.crossplane.io/v1alpha2
  kind: Object
  spec:
    providerConfigRef:
      name: kubernetes-default
    forProvider:
      manifest:
        apiVersion: apps/v1
        kind: Deployment
        metadata:
          namespace: default
          name: placeholder
        spec:
          replicas: 1
          selector:
            matchLabels:
              app: placeholder
          template:
            metadata:
              labels:
                app: placeholder
            spec:
              containers:
                - name: consumer
                  image: alpine:3.20
                  command:
                    - sh
                    - -c
                    - 'echo "Broker=$KAFKA_BROKER"'
                  envFrom:
                    - configMapRef:
                        name: placeholder
```



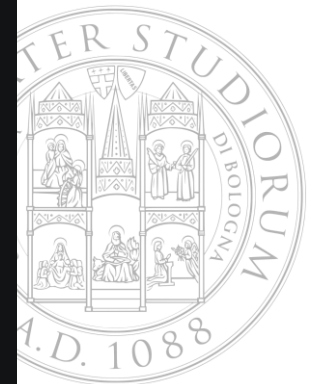
Kubebuilder implementation



datastream_types.go:

```
1 package v1alpha1
2
3 import (
4     metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
5     "k8s.io/apimachinery/pkg/runtime"
6 )
7
8 // DataStreamSpec defines the desired state of DataStream
9 type DataStreamSpec struct {
10     // TopicName is the name of the Kafka topic to manage.
11     // +kubebuilder:validation:MinLength=1
12     TopicName string `json:"topicName"`
13
14     // Partitions defines the total partition count for the topic.
15     // +kubebuilder:default:=1
16     // +kubebuilder:validation:Minimum=1
17     Partitions int32 `json:"partitions,omitEmpty"`
18
19     // ReplicationFactor defines the replication factor.
20     // +kubebuilder:default:=1
21     // +kubebuilder:validation:Minimum=1
22     ReplicationFactor int16 `json:"replicationFactor,omitEmpty"`
23 }
24
```

```
25 // DataStreamStatus defines the observed state of DataStream
26 type DataStreamStatus struct {
27     // Conditions track state transitions and lifecycle status.
28     Conditions []metav1.Condition `json:"conditions,omitEmpty"`
29
30     // TopicCreated confirms whether the Kafka topic exists on the broker.
31     TopicCreated bool `json:"topicCreated,omitEmpty"`
32
33     // ConfigMapRef holds the name of the generated connection details ConfigMap.
34     ConfigMapRef string `json:"configMapRef,omitEmpty"`
35 }
36
37 // +kubebuilder:object:root=true
38 // +kubebuilder:subresource:status
39
40 // DataStream is the Schema for the datastreams API
41 type DataStream struct {
42     metav1.TypeMeta `json:",inline"`
43     metav1.ObjectMeta `json:"metadata,omitEmpty"`
44
45     Spec DataStreamSpec `json:"spec"`
46     Status DataStreamStatus `json:"status,omitEmpty"`
47 }
48
49 // +kubebuilder:object:root=true
50
51 // DataStreamList contains a list of DataStream
52 type DataStreamList struct {
53     metav1.TypeMeta `json:",inline"`
54     metav1.ListMeta `json:"metadata,omitEmpty"`
55     Items []DataStream `json:"items"`
56 }
57
58 func init() {
59     SchemeBuilder.Register(func(s *runtime.Scheme) error {
60         s.AddKnownTypes(GroupVersion, &DataStream{}, &DataStreamList{})
61         metav1.AddToGroupVersion(s, GroupVersion)
62         return nil
63     })
64 }
```



Kubebuilder – Kafka topic creation

datastream_controller.go:

```
153 func createKafkaTopic(broker, topic string, partitions int32, reps int16) error {
154     conn, err := kafka.Dial("tcp", broker)
155     if err != nil {
156         return fmt.Errorf("dial broker failed: %w", err)
157     }
158     defer conn.Close()
159
160     controller, err := conn.Controller()
161     if err != nil {
162         return fmt.Errorf("cannot get kafka controller: %w", err)
163     }
164
165     controllerAddr := net.JoinHostPort(controller.Host, strconv.Itoa(controller.Port))
166     controllerConn, err := kafka.Dial("tcp", controllerAddr)
167     if err != nil {
168         controllerConn, err = kafka.Dial("tcp", broker)
169         if err != nil {
170             return fmt.Errorf("cannot dial kafka controller (%s) nor broker fallback (%s): %w", controllerAddr, broker, err)
171         }
172     }
173     defer controllerConn.Close()
174
175     err = controllerConn.CreateTopics(kafka.TopicConfig{
176         Topic:          topic,
177         NumPartitions:  int(partitions),
178         ReplicationFactor: int(reps),
179     })
180     if err != nil {
181         l := strings.ToLower(err.Error())
182         if strings.Contains(l, "already exists") || strings.Contains(l, "topic already exists") {
183             return nil
184         }
185         return err
186     }
187     return nil
188 }
```



Performance results

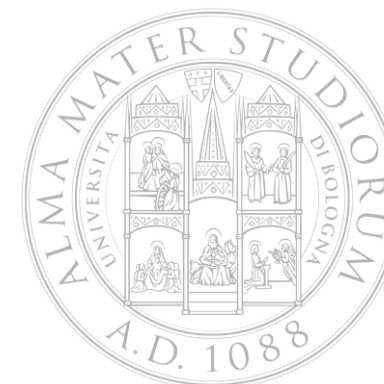
Metric	Crossplane	Kubebuilder
Lines of code written*	165	270
Edited files	7	2
Avg time to ready state (seconds)**	6	2

- Excluding generated code and provider installation/configuration manifests.

** The measured time is the elapsed time between applying the DataStream resource and observing the expected ready state.

Environment:

- Debian vps with 8 cores cpu, 8Gb ram, 75Gb ssd
- Local k3s cluster
- Single Kafka broker
- One DataStream resource per run
- Controllers/providers already running



REFERENCES

- The Kubebuilder Book, <https://book.kubebuilder.io>
- Crossplane documentation, <https://docs.crossplane.io/v2.3>
- kafka-go, <https://github.com/segmentio/kafka-go>



Questions?

Thank you for your attention

